

FACE ID

Hint: You can use GetHashCode(), Equals(), HashSet()

Problem Statement

Design and Analyze a Secure Identity Verification System Using Value-Based Equality in C#

Project Context

A security-sensitive application requires an **identity verification module** that can:

- Compare biometric facial data reliably
- Prevent duplicate identity registration
- Identify administrative users
- Correctly distinguish **value equality** from **reference equality**

Students must **implement, analyze, and explain** the behavior of each method involved in the authentication process.

1. Namespace Requirement

Namespace Name

Security.Authentication

All classes must be created inside this namespace.

2. Class Structure Overview

Class Name

Responsibility

FacialFeatures	Represents immutable biometric attributes
Identity	Represents a verified user identity
Authenticator	Handles registration and verification

TASK 1: FacialFeatures Class (Value Object)

Class Name

FacialFeatures

Data Members (Properties)

Name	Type	Access Modifier	Purpose
EyeColor	string	public (get-only)	Stores eye color
PhiltrumWidth	decimal	public (get-only)	Stores philtrum width

Constructor Task

FacialFeatures(string eyeColor, decimal philtrumWidth)

Students must:

- Assign the constructor parameters to the corresponding properties
- Ensure properties are immutable after object creation

Expected Outcome:

- FacialFeatures objects are immutable and safely comparable

Method-Level Tasks

Method: `Equals(Object obj) → bool`

Students must:

1. Check if both references point to the same object using `ReferenceEquals`
2. If not, attempt safe type casting to `FacialFeatures`
3. Delegate value comparison to the strongly-typed `Equals(FacialFeatures)`

Expected Outcome:

- Prevents unnecessary value comparison when references match
 - Ensures correct behavior when used in collections
-

Method: `Equals(FacialFeatures other) → bool`

Students must:

1. Check whether `other` is null
2. Compare `EyeColor` values
3. Compare `PhiltrumWidth` values
4. Return true only if all biometric values match

Expected Outcome:

- Two different objects with identical biometric data are treated as equal
-

Method: `GetHashCode() → int`

Students must:

1. Generate a hash code using **all value-based fields**
2. Ensure the hash code aligns with the `Equals()` logic

Expected Outcome:

- Enables correct behavior in HashSet and Dictionary collections

TASK 2: Identity Class (Composite Value Equality)

Class Name

Identity

Data Members (Properties)

Name	Type	Access Modifier	Purpose
Email	string	public (get-only)	Unique user identifier
FacialFeatures	FacialFeatures	public (get-only)	Biometric data

Constructor Task

Identity(string email, FacialFeatures facialFeatures)

Students must:

- Assign email and facial features to properties
- Ensure immutability after creation

Expected Outcome:

- Identity objects represent consistent user records
-

Method-Level Tasks

Method: `Equals(Object obj) → bool`

Students must:

1. Check reference equality using `ReferenceEquals`
2. Perform safe type casting to `Identity`
3. Delegate comparison to `Equals(Identity)`

Expected Outcome:

- Prevents incorrect equality results when compared with unrelated objects
-

Method: `Equals(Identity other) → bool`

Students must:

1. Check for null identity
2. Compare email addresses
3. Compare facial feature objects using their overridden `Equals()`
4. Return true only if both identity attributes match

Expected Outcome:

- Identity equality is determined by **email + biometric data**
-

Method: `GetHashCode() → int`

Students must:

1. Combine hash codes of Email and FacialFeatures
2. Ensure consistency with `Equals()`

Expected Outcome:

- Prevents duplicate entries in HashSet
-

TASK 3: Authenticator Class (Business Logic)

Class Name

Authenticator

Data Members

Name	Type	Access Modifier	Purpose
_registeredIdentities	HashSet<Identity>	private	Stores registered users
_admin	Identity	private static readonly	Fixed admin account

Method-Level Tasks

Method: AreSameFace(FacialFeatures faceA, FacialFeatures faceB) → bool

Students must:

- Compare facial biometric data using value equality
- Avoid reference comparison

Expected Outcome:

- Correct detection of identical facial features
-

Method: Register(Identity identity) → bool

Students must:

1. Check if the identity is already registered

2. Add identity only if it is unique
3. Return registration success status

Expected Outcome:

- Duplicate identities are rejected
-

Method: IsRegistered(Identity identity) → bool

Students must:

- Check existence of identity in the HashSet using overridden equality

Expected Outcome:

- Correct detection of registered users
-

Method: IsAdmin(Identity identity) → bool

Students must:

- Compare input identity with the fixed admin identity using value equality

Expected Outcome:

- Admin recognition works even for different object instances
-

Method: AreSameObject(Identity identityA, Identity identityB) → bool

Students must:

- Use `ReferenceEquals` to compare memory references only

Expected Outcome:

- Demonstrates reference equality vs value equality difference
-

TASK 4: Conceptual Analysis (Theory Task)

Students must explain:

Concept	Required Explanation
Value Equality	Comparison based on data
Reference Equality	Comparison based on memory
HashSet behavior	Why GetHashCode is mandatory
Immutability	Prevents hash corruption

Instructions

You are working on a system to simplify the login process for your organization's network. The tasks concern the authentication part. The system uses facial recognition to prove identity.

In all occurrences the eye color parameter is guaranteed to be non-null.

1. Ensure that facial features match

Implement `Authenticator.AreSameFace()` to check that two faces match.

Add equality routines for the `FacialFeatures` class.

```
Authenticator.AreSameFace(new FacialFeatures("green",
0.9m), new FacialFeatures("green", 0.9m);
// => true
Authenticator.AreSameFace(new FacialFeatures("blue",
0.9m), new FacialFeatures("green", 0.9m);
// => false
```

2. Authenticate the system administrator

Despite your protests the system administrator insists on having a built-in identity during acceptance testing so that they can always be authenticated.

The admin's email is `admin@exerc.ism`. They have green eyes and a philtrum with a width of 0.9.

Add equality routines for the `Identity` class.

Implement the `Authenticator.IsAdmin()` method to check that the identity passed in matches that of the administrator.

```
var authenticator = new Authenticator();
authenticator.IsAdmin(new Identity("admin@exerc.ism", new
FacialFeatures("green", 0.9m)));
// => true
```

```
authenticator.IsAdmin(new  
Identity("admin@thecompetition.com", new  
FacialFeatures("green", 0.9m)));  
// => false
```

3. Register new identities

Implement the `Authenticator.Register()` method which stores an identity on the authenticator itself such that calls to `IsRegistered()` will return `true` for this identity: otherwise `IsRegistered()` returns `false`.

To detect duplicated attempts to register an identity, if the identity has already been registered then `false` is returned by `Authenticator.Register()`, otherwise `true`.

```
var authenticator = new Authenticator();  
authenticator.Register(new  
Identity("tunde@thecompetition.com", new  
FacialFeatures("blue", 0.9m)));  
// => true  
authenticator.IsRegistered(new  
Identity("tunde@thecompetition.com", new  
FacialFeatures("blue", 0.9m)));  
// => true  
authenticator.Register(new  
Identity("tunde@thecompetition.com", new  
FacialFeatures("blue", 0.9m)));  
// => false
```

4. Prevent invalid identities being authenticated

Implement the `Authenticator.IsRegistered()` method and ensure it returns `false` when no identities have been registered.

```
var authenticator = new Authenticator();
authenticator.IsRegistered(new
Identity("alice@thecompetition.com", new
FacialFeatures("blue", 0.8m)));
// => false
```

5. Add diagnostics to detect multiple attempts to authenticate

A bug has been reported whereby the `Authenticator.IsRegistered()` method is called multiple times in quick succession for the same identity. You believe that there is some sort of "bounce" problem where the exact same record is being submitted multiple times. Your task is to add a diagnostic routine `Authenticator.AreSameObject()` to support any testing that's undertaken. The routine compares to objects and returns `true` if they are the exact same instance otherwise `false`.

```
var identityA = new Identity("alice@thecompetition.com",
new FacialFeatures("blue", 0.9m));
var identityB = identityA;
Authenticator.AreSameObject(identityA, identityB);
// => true
```

```
var identityC = new Identity("alice@thecompetition.com",
new FacialFeatures("blue", 0.9m));
var identityD = new Identity("alice@thecompetition.com",
new FacialFeatures("blue", 0.9m));
Authenticator.AreSameObject(identityC, identityD);
// => false
```

